Assignment No: 10

Title: Set up a CI/CD pipeline with Jenkins, Docker, and Kubernetes to automate the development-to-deployment process for your AI model

Steps:

Phase 1: Application Development & Local Containerization

1. Create the AI Application (app.py) Build a simple Flask API that represents your AI model's inference endpoint.

```
Python
from flask import Flask, jsonify import time

app = Flask(__name__)

@app.route('/predict', methods=['GET']) def predict():
 # Simulate a lightweight AI response
 return jsonify({
 "status": "success",
 "prediction": "Model output for Smart Surplus Connect",
 "timestamp": time.time()
 })

if __name__ == '__main__': app.run(host='0.0.0.0', port=5000)
```

2. Create the Dockerfile (Dockerfile) Define the environment needed to run your AI application.

```
Python
FROM python:3.9-slim
WORKDIR /app
RUN pip install flask COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

3. Build the Image Manually (Verification) Before automating, ensure the image builds correctly inside your local environment

```
None
# Point terminal to Minikube's Docker daemon eval $(minikube docker
env)

# Build the image docker build -t ai-cicd-app:latest .
```

Phase 2: Kubernetes Infrastructure Configuration

4. Deployment YAML (deployment.yaml) Define how many pods to run and specify the resource requests required for cluster management.

```
XML apiVersion: apps/v1
kind: Deployment
metadata:
 name: ai-cicd-deployment
spec:
 replicas: 2 selector:
matchLabels: app: ai-cicd
template: metadata:
labels:
 app: ai-cicd spec:
containers:
 - name: ai-container image: ai-cicd
app:latest imagePullPolicy: IfNotPresent
ports: - containerPort: 5000 resources:
requests: cpu: "100m" memory: "128Mi"
```

5. Service YAML (service.yaml) Expose the application so it is accessible within the cluster.

```
None apiVersion: v1
kind: Service
```

```
metadata:
 name: ai-cicd-service
spec: type:
NodePort selector:
app: ai-cicd ports:
 - port: 80 targetPort: 5000
nodePort: 30005
```

Phase 3: Jenkins Automation (The CI/CD Pipeline)

6. Define the Pipeline (Jenkinsfile) Create a script to automate the building of the Docker image and the deployment to Kubernetes.

```
None pipeline {
agent any stages
{
stage('Checkout') {
 steps {
 // Pull code from your repository
 checkout scm
 } }
 stage('Build Docker Image') {
 steps {
 // Build the image using the local Docker daemon sh 'docker build -t ai-cicd-app:latest .'
 } }
 stage('K8s Deployment') {
 steps {
 // Apply the configurations to the Kubernetes cluster sh 'kubectl apply -f deployment.yaml' sh
'kubectl apply -f service.yaml'
 }
 }
 }
}
```

Phase 4: Monitoring and Execution

7. Run the Pipeline * Open Jenkins and create a **New Item** -> **Pipeline**.

- Link your Git repository containing the files above.
- Click **Build Now** to trigger the automation.

8. Verify Deployment and Health Check the status of your pods and service to ensure the automation succeeded

None

Check if pods are running kubectl get pods

Get the URL to access your AI model

minikube service ai-cicd-service --url **Outputs :**

